

The Omega proof: genesis-anchored ZK attestation of any Cardano claim

The Omega proof: genesis-anchored ZK attestation of any Cardano claim

Status: draft design with measured feasibility. This page answers one question: is it possible to generate a proof about any of the [42 provable Cardano properties](#) such that a verifier can validate it against all epochs from genesis to the time the proof was committed? The answer is yes. The per-block circuit cost is measured, not estimated (see the Phase 0 spike in §6). Chain statistics cited here were live-fetched 2026-07-15 (Koios / AdaStat, mainnet epoch 643); they are session research, not yet verbatim-gated KB pages.

1. Thesis

The [42-claim catalog](#) grades every claim by its anchor, meaning the commitment the proof is checked against. Today's practical anchors each attest a snapshot that you trust because a stake quorum signed it: a [Mithril certificate](#), a [Peras certificate](#), or a [CIP-0165 SCLS root](#). The Omega proof replaces the snapshot anchor with something that needs no signer set at all:

One recursively maintained accumulator digest Ω that provably summarizes the entire Cardano chain, headers verified and ledger state re-derived, from the genesis block to the moment the proof lands on-chain. Any of the 42 claims, about any epoch in history, is then a short Merkle lookup against Ω .

Two words carry most of this page and deserve plain definitions up front. "The fold" is a replay of the whole chain inside a proof system: a program walks every block from genesis, checks the consensus signatures, applies the transactions to its own copy of the ledger state, and emits a proof that it did so correctly. "IVC" (incrementally verifiable computation) is the technique that keeps this affordable: each step proves "the previous step's proof was valid, and I extended it by one block," so the proof stays small while the history it covers grows without bound. Circuit sizes on this page are quoted as K , where $K = \lceil \log_2(\text{constraints}) \rceil$, the exponent of the power-of-two domain the proof system needs for a circuit of that size.

Three facts make this buildable now:

1. **Mina precedent.** A whole proof-of-stake chain can be compressed into one recursive SNARK; IVC is deployed technology, not a research bet. Midnight's own stack does recursion by [in-circuit KZG verification](#) (128-bit truncated Fiat-Shamir challenges, aPLONK committed instances). No curve cycle is needed (some recursion schemes, Mina's included, need a matched pair of curves that embed each other; the Midnight stack avoids that requirement).

2. **The Cardano landing is proven live.** A commitment-Groth16 verifier already [runs on Cardano Preview](#) over the [CIP-0381](#) builtins at 32 to 39% of one transaction budget, with 336-byte proofs and one public input.
3. **The per-block header circuit is the size of a circuit already proven in 43 seconds.** The Phase 0 spike (§6) measured the full Praos header validity check at 3,514,285 R1CS constraints ($K=22$), within 2% of the deployed recovery circuit (3,450,403, $K=22$) that proves in 43 s on a 24 GB machine. Body application adds a further 1 to 3 times that per block (§6), which is still one-machine territory with GPU-class hardware.

2. The problem Omega must solve: Cardano headers commit no state

Verified against the ledger CDDL (shelley/babbage/conway): a Cardano header carries `prev_hash`, `block_body_hash`, the VRF result, the KES signature and operational certificate, and the protocol version. There is no state root. Unlike Ethereum or Mina, nothing in a Cardano header commits to the UTxO set, the stake distribution, reward accounts, or governance state. All of that is derived by folding the ledger rules over every transaction since genesis. This is why [Mithril](#) exists, and why [CIP-0165](#) proposes a canonical ledger-state commitment going forward.

Two consequences shape the whole design:

- **The fold must carry the state itself.** A header-only light client cannot even validate headers, because the VRF check needs the epoch's stake distribution and nonce, and those come from block bodies. So the Omega accumulator re-derives ledger state in-circuit and commits to it. That is the dominant cost. It is also the payoff (§4): every claim the catalog grades "hard, needs ledger replay or a closed authenticated range" becomes a cheap lookup against state the fold already derived once, amortized over all claims forever.
- **History is finite and known.** Mainnet at epoch 643 (2026-07-15): 13,681,784 blocks, split into 4.49M Byron headers and 9.19M Shelley-family headers (TPraos through epoch 364, then Praos), across 435 Shelley-family epochs, roughly 220 GB of chain, about 122.4M transactions, about 2,900 active SPOs, and about 1.35M delegators. These numbers bound the one-time backfill (§6).

3. Architecture: four layers

3.1 Layer 1: the chain-fold IVC (the Ω -accumulator)

An IVC over every block since genesis, built on the [Halo2/Plonkish](#) stack with [midnight-proofs recursion](#) (KZG over BLS12-381, the same [unified substrate](#) as the bridge). The carried state Σ is a bundle of commitment roots:

```

Σ = { utxo_root      : SMT over created outputs, with spent markers
      (gives membership AND nonmembership/spent-status)
      tx_mmr        : our own MMR of tx ids per block (block_body_hash is a
                    flat hash, not proof-friendly; we build the tree Cardano didn't)
      stake_mark/set/go : the three stake-snapshot roots (set gates the VRF check)
      pool_root      : pool registrations incl. VRF key hashes (header checks need them)
      reward_root    : reward accounts; reward_event_mmr : reward/MIR/treasury events
      asset_root     : cumulative mint/burn accounting per (policy, asset)
      params_hash, era_id, nonce_state (evolving/candidate nonces)
      gov_root       : Conway governance state (proposals, votes, DReps, committee)
      epoch_mmr      : MMR of per-epoch bundles E_e (see below) }

```

Two tree shapes recur in Σ and do different jobs. An SMT (sparse Merkle tree) is keyed by identity (an output reference, a stake credential), holds a leaf for every possible key, and therefore supports proofs of absence as well as presence; that is what makes UTxO nonmembership provable. An MMR (Merkle mountain range) is an append-only accumulator that grows cheaply and never rewrites old leaves; that fits transaction logs and the per-epoch bundles, where history only ever extends.

Hash discipline (driven by the measured costs in §6): one Blake2b-256 compression costs about 33k R1CS and one SHA-512 compression about 128k. So every Omega-internal structure (SMTs, MMRs, epoch bundles) uses an algebraic hash (Poseidon-class over the BLS12-381 scalar field, roughly two orders of magnitude cheaper). The bit-faithful Blake2b/SHA-512 price is paid only where Cardano consensus fixes the hash: header hashes, tx ids, credentials, and the VRF/KES internals.

Per-block step circuit s_{block} (measured: 3.51M R1CS, K=22, §6):

1. chain link: `prev_hash` = Blake2b-256(previous header);
2. era-dependent VRF check: ECVRF-ED25519-SHA512-Elligator2 verify (older TPraos headers, epochs 208 to 364, carry separate leader and nonce VRF certificates, so two verifies; Babbage merged them into one), input Blake2b(slot || η_e), where η_e is the epoch nonce, the shared randomness each epoch distills from VRF outputs, plus the leader-value and nonce-contribution derivations and the stake-threshold comparison against the pool's σ in `stake_set`;
3. Sum6 KES verification of the body signature (one Ed25519 leaf verify plus a 6-level Blake2b vkey tree, not seven signature checks);
4. operational certificate: Ed25519 verify under the pool cold key registered in `pool_root`, opcert counter monotonicity;
5. body application: parse the block body against `block_body_hash`, hash tx ids (Blake2b, consensus-fixed), apply inputs/outputs/certs/mint/withdrawals to the Σ roots (Poseidon SMT updates), fold the VRF output into `nonce_state`, and append tx ids to `tx_mmr` together with each transaction's validity flag (the Alonzo-era `invalid_transactions` marking), which is what lets script-outcome claims (§4) later prove a transaction was accepted as valid.

Per-epoch circuit s_{epoch} : snapshot rotation (mark→set→go: Cardano keeps three lagged copies of the stake distribution, where mark is the newest, set is the one leader election reads, and go is the one rewards read; each epoch boundary shifts them one position), the reward calculation (unavoidable: rewards compound into stake, and stake gates the VRF threshold, so an Omega that skipped rewards could not validate the next epoch's headers), nonce finalization $\eta_{\{e+1\}} = \text{Blake2b}(\text{candidate} \parallel \text{lastBlockNonce})$,

protocol-parameter and era transitions, then append the epoch bundle $E_e = \text{Poseidon}(e, \eta_e, \text{all } \Sigma \text{ roots})$ to `epoch_mmr`. The epoch MMR is the "all epochs from genesis" mechanism: one digest, any historical epoch extractable with a logarithmic path. Bundles snapshot state at epoch boundaries; facts that change mid-epoch remain provable because the events themselves live in `tx_mmr` with their block positions, and the fold can also append per-block bundles (one Poseidon hash per block, negligible) for applications that need state reads at slot resolution.

Byron prologue: a separate, cheaper step circuit (about 1.2M R1CS from the same components): Ed25519 signature against the federated genesis delegation, Blake2b chain links, epoch-boundary-block handling, no VRF/KES. The alternative is a pinned checkpoint past Byron, which is what Ouroboros Genesis deployments do. That is weaker, and unnecessary given the modest cost.

Backfill and steady state: IVC alone is sequential (each step needs the previous state), but the backfill parallelizes as proof-carrying data: chunk the chain, prove chunks independently, then binary-merge. This composes soundly because every chunk proof exposes its start and end states ($\Sigma_{\text{in}}, \Sigma_{\text{out}}$) as public values; the merge circuit accepts two chunks only when the first chunk's Σ_{out} equals the second chunk's Σ_{in} , and the leftmost chunk must start from the fixed genesis state. A fabricated intermediate state cannot survive the merges back to genesis. The backfill is a one-time compute (§6). At the tip, one block arrives every ~20 s on average, and one full S_{block} step (header check plus body application) is roughly 7M to 14M R1CS, so a single GPU-class prover, or a short pipeline of CPU machines, keeps the accumulator current in real time.

3.2 Layer 2: the claim layer (where "Omega" universality lives)

The universality is in the accumulator, not in a mega-circuit. Each of the 42 claims gets a small claim circuit C_j whose public inputs are the [claim envelope](#) and the Ω -digest, and whose witness is: an `epoch_mmr` path $\rightarrow$ epoch bundle $E_e \rightarrow$ Merkle/SMT path(s) into the relevant root(s) $\rightarrow$ the claim-specific predicate. Claim circuits are small (tens of thousands to a few million constraints, K of about 15 to 22). Dispatch is exactly the [claim-interface-schema](#) verifier registry: `predicateId` maps to a pinned VK, and the Omega anchor slots into the existing envelope as `anchor_type = omega_accumulator, anchor_digest =  $\Omega$` .

3.3 Layer 3: the landing wrap

Two different Halo2 proofs need to land on Cardano, and they land at different times through the same machinery (§9.1 walks this in detail):

- **The fold proof lands once per checkpoint.** A [commitment-Groth16](#) wrapper proves "a valid Halo2 chain-fold proof exists that extends the previously committed digest Ω_{prev} to Ω_{new} "; the checkpoint transaction verifies it and records Ω_{new} . No later transaction re-verifies the fold.
- **A claim proof lands per consuming transaction.** A second wrapper proves "a valid Halo2 claim proof against Ω exists", with public input `pub =  $H(\text{envelope} \parallel \Omega)$` , where the validator reads Ω from the committed checkpoint rather than from the prover.

Both wrappers extend the live [Preview deployment's](#) `groth16VerifyCommitted` (measured 3.2 to 3.9×10^9 ExCPU, or 32 to 39% of one tx budget). The open cost item for both is the Halo2-verifier-inside-Groth16 circuit (§11); the fallback is IOG's [Halo2-Plutus verifier](#) directly (no wrap, no per-circuit setup, the same trade

the [bridge design](#) weighs for its Midnight-to-Cardano landing). On Midnight, both proofs verify natively and no wrap exists at all. Both wrappers must be frozen circuits so their Groth16 trusted setup is a once-ever event rather than a per-change one; §9.5 works through how the fold's era-specific and claim-specific logic stays inside the recursion so the wrap constraint system never changes.

3.4 Layer 4: self-checkpointing (closing the "time of commitment")

Every Omega checkpoint committed on Cardano pins its Ω -digest on-chain, and the next checkpoint must extend from the last committed digest. Cardano thereby becomes the checkpoint ledger for the proof of its own history.

The eUTxO mechanics that make the lineage unique and forgery-proof: the checkpoint lives in a single UTxO identified by a thread NFT (a one-shot minting policy, so exactly one such token can ever exist). Advancing the checkpoint means spending that UTxO and re-creating it with the new datum. The checkpoint validator enforces continuity: the fold proof it verifies exposes its start and end digests (Ω_{prev} , Ω_{new}) as public values, and the validator requires Ω_{prev} to equal the digest in the datum being spent. Submission is permissionless, since anyone holding a valid fold proof can advance the thread; correctness is gated by the proof, not by the submitter's identity. Two competing lineages cannot arise because there is one thread token, and consumers of claims locate the checkpoint by that token rather than by address. Combined with folding only settled blocks (§9.4), this makes "genesis to the time the proof was committed" literal: the commitment transaction itself closes the interval, and a verifier of any later claim walks the on-chain digest lineage instead of trusting a prover-supplied tip.

4. Coverage: all 42 claims against Σ

Every claim in the catalog reads one or more Σ roots via an epoch bundle. The formerly-"hard" rows are the point: the expensive ledger replay is done once inside the fold and amortized over every claim forever.

Claims (catalog IDs)	Group	Σ root(s) consumed	Notes
1 to 4	Chain and epoch context	<code>epoch_mmr</code> bundle fields (era_id, params_hash, η_e)	direct bundle reads
5 to 9	Tx and block facts	<code>tx_mmr</code> (plus witnessed tx body against its Blake2b tx id)	Omega builds the per-tx tree Cardano's flat <code>block_body_hash</code> never gave us
10	UTxO membership	<code>utxo_root</code>	SMT membership
11	UTxO nonmembership / spent status	<code>utxo_root</code> spent markers	catalog grade "hard, needs closed authenticated range" becomes free: the fold derived the full set
12 to 15	Output address/value/datum/script	<code>utxo_root</code> leaf = $H(\text{outref} \parallel \text{output})$	leaf opening
16, 18	Mint/burn, metadata	<code>tx_mmr</code> plus tx body	tx-level facts
17	Asset supply at epoch E	<code>asset_root</code>	catalog grade "derived/hard" becomes a running total maintained by the fold
19 to 25	Script outcomes (all 7 purposes)	<code>tx_mmr</code> plus block validity	acceptance-implies-valid under Model B (§5): a tx in a canonical block and not in <code>invalid_transactions</code> had all scripts pass; per-claim replay = Model A opt-in
26 to 32	Stake, delegation, pools	<code>stake_*</code> , <code>pool_root</code>	maintained anyway; the VRF check needs them
33 to 37	Rewards, deposits, treasury, MIR	<code>reward_root</code> , <code>reward_event_mmr</code>	catalog grade "hard; spans epochs" becomes a natural byproduct of <code>S_epoch</code> 's mandatory reward fold
38 to 42	Governance (CIP-1694)	<code>gov_root</code> plus epoch-boundary bundles	ratification/expiry/enactment are epoch-boundary state transitions the fold already executes

This 42-claim catalog is the Cardano half of a two-sided bridge. The Midnight half, the 52 predicates provable about Midnight's own state, is catalogued in §12, where the key point is that Midnight needs no accumulator like Ω at all.

5. Security models: full replay vs consensus-anchored

Model A, full validation replay. The fold re-executes phase-1 ledger rules and phase-2 Plutus evaluation (an untyped-Plutus-Core CEK machine in-circuit, in other words a Plutus zkVM). Strongest possible statement: "this chain is valid under the ledger rules, full stop." The cost is a zkVM for UPLC plus cost-model accounting across 10 protocol versions. That multiplies proving cost severalfold and is the single largest engineering item. Not recommended as the base.

Model B, consensus-anchored (recommended). The fold verifies everything consensus-cryptographic: header chain, VRF against stake and nonce, KES, opcerts, and era and parameter transitions. It then applies block bodies without re-running Plutus. Canonicity (that the folded chain is *the* chain, not merely *a* valid chain) comes from two mechanisms working together. First, the fold consumes only blocks already settled under Cardano's own rules (§9.4). Second, checkpoint continuity (§3.4) plus the fact that the fold derives the

stake distribution itself means an attacker who wants to extend the committed lineage with a fabricated fork must present blocks whose VRF leadership verifies against the very stake distribution the fold derived, which requires the same adversarial stake that Praos already defends against. The residual gap is the classic long-range scenario (expired keys from old epochs), which the settled-prefix rule blunts and an in-circuit density rule in the style of Ouroboros Genesis would close; the exact formulation is under study (§11). The soundness assumption is honest stake majority, that is, the canonical chain contains only blocks honest nodes fully validated. This is the same assumption every Cardano full node already makes when it selects the densest or longest valid chain, so Omega adds no new trust. Script-outcome claims come free via acceptance-implies-valid (§4, claims 19 to 25).

The recommendation: Model B as the accumulator's base, Model A as a per-claim opt-in. When a consumer demands it, a claim circuit can re-execute one transaction's scripts (bounded, known cost) against inputs proven from `utxo_root`, without ever paying for a full-chain zkVM.

	Model A (full replay)	Model B (consensus-anchored)
Statement	chain valid under ledger rules	chain canonical under honest-majority consensus
Extra assumption	none beyond crypto	honest stake majority (same as every node)
Plutus zkVM	required, full-chain	none (opt-in per claim)
Relative fold cost	several ×	1× (measured, §6)

6. K complexity: measured, then totaled

Phase 0 spike: `spikes/omega-header-circuit` (private workspace; results file `RESULTS.md`), gnark v0.15.0 over the BLS12-381 scalar field, reusing the audited proof-zk-recovery gadgets (emulated Ed25519, SHA-512, Blake2b) plus new variable-base scalar-mult, ECVRF (draft-03 including Elligator2), Sum6 KES and RFC-8032 gadgets. Compile-only counts (structurally faithful, not yet test-vectored); $K = \lceil \log_2(\text{constraints}) \rceil$.

circuit	R1CS	K	PLONKish (scs)	K
SHA-512, one block region	256,668	18	799,300	20
Blake2b-256, 896-byte header	230,189	18	760,935	20
[s]B fixed-base (windowed)	60,889	16	215,994	18
[s]P variable-base, 256-bit	370,440	19	1,273,989	21
Ed25519 verify (opcert shape)	978,410	20	3,308,257	22
ECVRF verify (full, incl. β)	1,147,114	21	3,834,727	22
Sum6 KES verify (header msg)	1,539,233	21	5,100,199	23
S_block composed (Babbage+)	3,514,285	22	11,859,251	24

Derived figures (estimates around the measured core):

- TPraos step (epochs 208 to 364, two VRF certs): add about 1.15M, for about 4.7M R1CS.
- Byron step: about 1.2M R1CS (Ed25519 plus Blake2b only).

- Body application (Model B): tx-id hashing is consensus-Blake2b over an average tx of about 1.8 KB (about 14 compressions, 0.47M R1CS per tx); Σ -root updates are Poseidon SMT paths (10k to 30k R1CS per update). Body work lands at roughly 1 to 3 times the header check per block and, over the 122.4M historical transactions, about 2^{45} to 2^{46} total.
- S_epoch reward fold: per-delegator Poseidon updates over about 1.35M delegators come to 2^{36} to 2^{38} per epoch, tree-split into $K=22$ sub-proofs, or about 2^{45} to 2^{46} over 435 epochs. Co-dominant with headers, and unavoidable (stake gates VRF).
- Recursion overhead: in-circuit KZG verification per step ([midnight-proofs recursion](#)), about 2^{17} to 2^{19} rows. Noise against a 12M-row step. That figure is an accumulation cost, not a pairing: the scheme defers the pairing check, folding each step's openings into an accumulator, and discharges a single final pairing when the fold proof is prepared for landing. No per-step pairing is evaluated inside a circuit.
- Claim circuits: $K = 15$ to 22 (epoch-MMR path plus SMT openings plus the predicate).
- Groth16 landing wrap (Halo2 verifier in a Groth16 circuit): estimated 20M to 40M constraints, $K = 25$ to 26 . This is the riskiest unmeasured item (§11), with the direct Halo2-Plutus landing as fallback.

Totals. Headers: $9.19M \times 3.5$ to $4.7M$ plus $4.49M \times 1.2M$, about 2^{45} . Bodies: about 2^{45} to 2^{46} . Rewards: about 2^{45} to 2^{46} . The grand total is roughly 2^{46} to 2^{47} constraint-work from genesis to epoch 643, compressed by IVC/PCD into one proof whose verifier is $O(1)$ regardless of history length: one Groth16 verify on Plutus V3, already measured live at 32 to 39% of a transaction budget.

Proving economics. Backfill is roughly 0.7 to 1.4×10^{14} constraints. At 10^6 to 10^7 constraints per second per GPU-class worker, that is 100 to 1,600 GPU-days, and the work is embarrassingly parallel (a 100-worker fleet finishes in days to weeks, once). Steady state is the full S_block step, header check plus body application, roughly 7M to 14M R1CS per ~20 s block: two to four times the recovery circuit's measured 43 s CPU prove, so one GPU-class box, or a short pipeline of CPU machines, holds the tip. Epoch boundaries add a burst once per 5 days.

7. What lands on-chain

The same shape as the [bridge](#) Midnight-to-Cardano landing: a 336-byte commitment-Groth16 proof, one public input `pub = H(claim envelope ||  $\Omega$ )`, and a validator that reconstructs `pub` itself ([zk-recovery architecture](#) discipline: never trust a prover-supplied public input), a VK pinned as an immutable parameter, a nullifier spent-map for consumable claims (a nullifier is a one-time tag derived from the claim; recording it stops the same claim being consumed twice, and each application keeps its own spent-map), plus the Layer-4 digest-lineage datum. Budget: verify costs about 32%, the full claim-consume branch 39 to 55% as measured on Preview, and batching amortizes to roughly 8 to 15 claims per transaction.

8. Applications and user experience

The design so far describes machinery. This section describes what the machinery is for, and what it feels like to use. One structural fact drives all of the user experience: the expensive part (the chain fold) is shared public infrastructure, maintained once for everyone, while the part a user runs (a claim circuit, K of 15 to 22) proves in seconds on consumer hardware. Users never touch the fold. They fetch the current Ω and a witness path

from an indexer they do not need to trust, prove their small claim locally, and hand over a proof that verifies in one shot.

8.1 Application domains

Trustless bridge transfers (claims 1 to 9 plus finality). The founding use case. A user locks ADA or cNIGHT at the bridge script on Cardano; a prover attests the lock UTxO against Ω ; Midnight mints the wrapped asset. Today's federated `c2m-bridge` asks users to trust a committee that approves transaction hashes. With Omega the flow looks the same from the outside (send, wait, receive), but the waiting is settlement plus proving latency rather than committee sign-off, and once the fold is genesis-anchored (Phase 2, §10) there is no committee to compromise. Honest numbers for the wait: the fold only consumes settled blocks (§9.4), so a fresh lock becomes provable in minutes once Peras certificates are live, but roughly 12 hours under today's plain Praos settlement depth. The wallet can show plain stages: confirming on Cardano, settling, proving, minted.

Private credentials from chain history (claims 26 to 37, plus 10 and 12 to 15). This is where the catalog and Midnight's selective-disclosure model reinforce each other. The address, the amounts, and the transaction ids sit in the witness; only the predicate is public. So a user can prove statements like "I delegated at least 10,000 ADA to pool P for epochs 400 through 500" or "this wallet has earned staking rewards continuously since 2021" without revealing which wallet they mean. Products this enables:

- airdrop and ISPO eligibility checks that do not ask anyone to paste an address into a form;
- loyalty tiers for long-term delegators, provable rather than asserted;
- sybil-resistant reputation ("wallet older than epoch 300 with continuous activity") for governance weighting or allowlists;
- DAO membership gated on historical holdings without doxxing members.

The proof is generated inside the wallet. To the user it looks like a "prove eligibility" button followed by a QR code or a small file. To the verifier it looks like a signature check: milliseconds, no callback to any server. The comparison that fits best is OAuth, except the only thing that leaves the user's device is the yes.

Proof of reserves and supply audits (claims 10 to 15 and 17). An exchange or a stablecoin issuer proves "these outputs held X ADA at epoch E and were unspent" on a schedule. Claim 11 (nonmembership and spent status) is what makes such an attestation honest: without it, a reserves proof can double-count outputs that were already spent. The catalog graded claim 11 hard for exactly this reason, and the Omega fold makes it a plain lookup. Claim 17 adds provable circulating supply per asset policy. For the institution the experience is a scheduled job that publishes one proof per epoch. For the public it is a dashboard tile whose caption changes from "attested by an auditor" to "verified against the chain from genesis."

Notarization and timestamping (claims 6 and 18). Prove that a metadata hash was anchored in a transaction at slot S. Because the anchor is Ω rather than a service, the proof stays checkable decades later by anyone with the verifier and the on-chain digest lineage: no node, no explorer, no archive service. Uses include legal evidence, publication priority, and supply-chain records. The experience: drop a file in, get a proof file out, and anyone can check it in a browser in milliseconds. One caveat separates checking from creating: an existing proof verifies forever with nothing but the verifier and the digest lineage, but generating

a new proof about old history needs the witness trees that indexers maintain (§9.3), so archival proving depends on someone keeping that data alive.

Governance accountability (claims 38 to 42). Prove a DRep's complete voting record; prove that you voted, or abstained, without revealing your stake; let a Midnight contract or a partner chain react automatically to an enacted Cardano governance action, for instance releasing matching funds when a treasury withdrawal is ratified. Cross-chain governance becomes event-driven: the consuming contract verifies the governance fact itself instead of polling an oracle.

A ZK coprocessor for Plutus (any claim, consumed on-chain). Plutus validators run under tight budgets and cannot see history. With the landing verifier (§7), a validator can consume any historical fact for 32 to 39% of one transaction budget: insurance contracts settling on proven historical protocol parameters, derivatives referencing proven epoch-boundary stake distributions, lending protocols pricing against a wallet's proven history. The [claim envelope](#) already defines the interface a validator sees: `predicateId`, context binding, nullifier, expiry. This is the same pattern Ethereum projects such as Axiom and Herodotus sell as "ZK coprocessing," here with a genesis-anchored rather than contract-storage anchor.

Genesis-grade light clients and untrusted indexers. A phone wallet or an embedded device verifies one proof instead of syncing 220 GB or trusting a hosted API. Sync takes seconds and the security model matches a full node under Model B's assumption. The quieter consequence is what it does to infrastructure trust: today nearly every "decentralized" Cardano application trusts Blockfrost or Koios to tell it the truth. Against Ω , an indexer becomes a convenience that serves data plus witness paths, and everything it serves is checkable. A lying indexer can only fail to answer; it cannot deceive.

Partner-chain bootstrapping. Any new chain gets Cardano finality and state with one verifier contract, no federation phase, by verifying the fold proof (or its wrap) natively. Midnight is the first consumer; nothing in the design is Midnight-specific on the consuming side.

8.2 The experience, by role

End users mostly never learn any of this exists. Proofs hide behind actions they already perform: bridging, claiming, checking in. Three latencies are visible if you look. First, anchor freshness: the fold trails Cardano settlement (§9.4), so a claim about something that happened moments ago waits minutes once Peras certificates are live, or around 12 hours under today's plain Praos depth; claims about anything historical have no wait at all. Second, claim proving: circuits at K of 15 to 22 prove in seconds on a laptop or a recent phone, and a wallet can also delegate proving to a service when the witness is not private. Third, the Groth16 wrap, when the claim lands on Cardano: minutes at a wrapping service (§9.1), which a wallet surfaces as a pending state. Off-chain and Midnight-side verification skip the third wait entirely. In practical terms: credentials feel like Face ID followed by a share sheet, and an on-chain claim feels like a slightly slow transaction.

Prover operators are the new infrastructure role, comparable to running a Mithril aggregator or an L2 prover. The job: run the one-time backfill (a fleet burst, §6), then keep pace with one `S_block` step per 20-second block and an `S_epoch` burst every 5 days. The trust story is what makes the role easy to decentralize: a wrong fold does not verify, so operators cannot cheat anyone. Several can run side by side, and users switch by changing a URL. Who funds the role is an open design question (§11): candidates include protocol or treasury

funding as public infrastructure, fees from wrapping and checkpoint services, and applications sponsoring the checkpoint cadence they need.

Application developers see an SDK and a registry. The write path is one call, `prove(predicateId, params)`, returning an envelope and a proof; the read path is a registry lookup plus a verify. It feels like calling an indexer API, except the response is portable, permanent, and carries its own verification. The anchor field in the envelope also gives developers a low-risk adoption path: ship today against a Mithril anchor (`anchor_type = mithril_certificate`, the Phase 1 configuration in §10), then flip to `anchor_type = omega_accumulator` when the genesis-anchored fold is live. No user-facing flow changes; the trust model underneath gets stronger. Be precise with users about what each anchor means: a Mithril anchor still trusts a stake-quorum signer set, so the "no committee at all" property arrives with Phase 2 (§10), not before.

Verifiers have the shortest description: one Groth16 check. On Cardano that is a third of a transaction budget; on Midnight it is native; in a browser or a phone it is milliseconds. A verifier needs the pinned VK, the registry entry, and the on-chain digest lineage, and nothing else.

8.3 A worked example: private delegation credential

To make the pipeline concrete, the full path for one flagship flow, a delegation-history credential used for an airdrop:

1. A project publishes predicate `delegated_epochs(pool, min_stake, e_from, e_to)` in the registry, pinned to a VK.
2. The user's wallet fetches the current Ω , the `epoch_mmr` paths for epochs 400 through 500, and the `stake_set` SMT openings for its own stake credential, all from an untrusted indexer.
3. The wallet proves the claim circuit locally (seconds): the openings chain up to Ω , the stake amounts clear the threshold in every epoch, and the stake credential stays in the witness.
4. The wallet wraps the claim in an envelope: `predicateId`, `anchor_digest =  $\Omega$` , a context hash binding the airdrop campaign, a nullifier so the credential cannot be claimed twice, and an expiry.
5. The project's validator (or off-chain checker) verifies the proof against the registry VK and the on-chain Ω lineage, marks the nullifier spent, and pays out. Where the nullifier gets recorded depends on the consumer: an on-chain validator keeps a spent-map in its own state (a per-application structure, shardable with the usual eUTxO concurrency patterns), while an off-chain checker records it in the campaign's own database. Same envelope, different ledger.

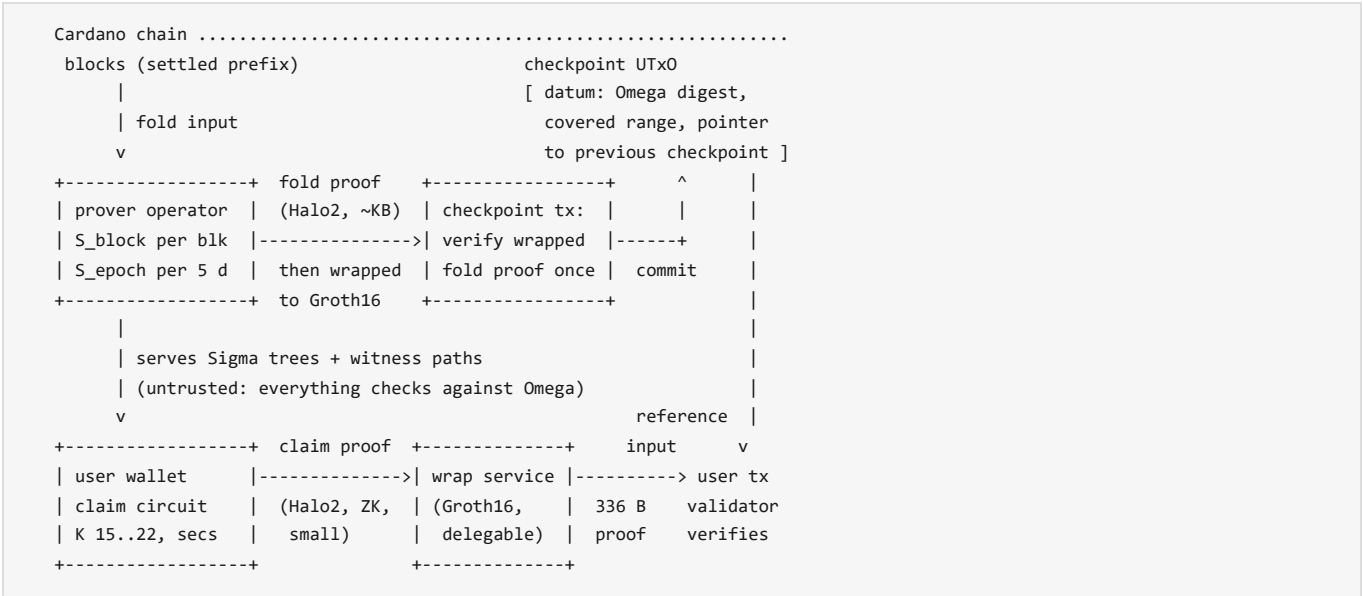
The user pressed one button and revealed one bit. The project learned nothing about the wallet except that it qualifies, and it never operated or trusted an indexer, an oracle, or a KYC vendor to establish that fact.

9. Where the proofs live and when they update

The architecture (§3) says what gets proven. This section explains the operational side: who produces each proof, where each artifact is stored, and on what schedule things move. The questions it answers come up first whenever someone tries to build against the design: does a user generate the Groth16 proof or fetch it from somewhere, where does the chain-fold proof physically live, and how often does anything need updating.

9.1 Three artifacts, three producers

A transaction that consumes a historical claim involves three distinct proof artifacts. They have different producers, different sizes, and different lifetimes, and conflating them is the most common way to misread the design.



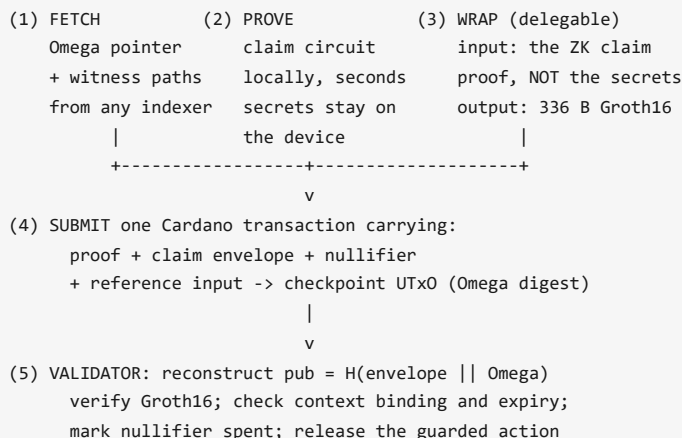
The chain-fold proof is referenced, never user-generated. The recursive Halo2 proof that Ω is correct is produced continuously by prover operators (§9.4). Cardano cannot verify Halo2 natively, so for the checkpoint landing the operator wraps the fold proof in the same commitment-Groth16 landing the claims use (§3.3), or verifies it via the direct Halo2-Plutus route if that wins the §11 comparison. The wrapped proof exposes the fold's start and end digests (Ω_{prev} , Ω_{new}); the checkpoint transaction verifies it once and the checkpoint validator requires Ω_{prev} to match the datum being spent (§3.4). After that transaction settles, the chain holds the digest and nobody needs the fold proof again to consume claims. A user's transaction reaches the digest through a reference input (CIP-31): it reads the checkpoint datum without spending it, so any number of claims can share one checkpoint.

The claim proof is generated by the user, locally. It is the small Halo2 circuit from §3.2 (K of 15 to 22, seconds on consumer hardware). It has to be generated by the user because its witness can contain secrets: a stake credential, output details, amounts. The witness paths it opens (epoch MMR, SMT openings) come from any indexer, and the indexer needs no trust because every path either chains up to Ω or the proof fails.

The Groth16 wrap is produced per transaction, usually by a service. The wrap circuit (estimated K of 25 to 26) needs tens of gigabytes of RAM and minutes of proving, which is not phone territory. Delegating it is safe for two reasons. First, the wrap's witness is the Halo2 claim proof itself, not the user's secrets, and that claim proof is already zero-knowledge, so the service learns only the public claim statement the verifier would see anyway. Second, the claim envelope's `context_hash` binds the claim to the user's specific transaction inside the proof (the same discipline the [recovery circuit](#) uses to bind the destination address), so a wrapping service that turns malicious can censor a user but cannot redirect or steal the claim. If the direct [Halo2-Plutus verifier](#)

route wins the open comparison in §11, this third artifact disappears and the user's own claim proof lands on-chain as-is, at higher ex-unit cost but with no delegation at all.

9.2 A spend against history, step by step



Step 5 repeats the on-chain discipline from §7: the validator reconstructs the public input from data it already trusts (the checkpoint datum, the envelope fields, its own parameters) rather than accepting a prover-supplied value, verifies against a VK pinned at deployment, and records the nullifier so a consumable claim spends once. Batching changes nothing structurally: an aggregator can wrap 8 to 15 claims into one proof and one transaction, and each claim keeps its own envelope, context binding, and nullifier.

9.3 What is stored where

On-chain: the digest, not the proof. A checkpoint UTxO carries a small datum: the Ω digest, the slot range it covers, and a pointer to the previous checkpoint. The fold proof that justified it rode in the committing transaction and was verified once by the checkpoint validator; the chain does not need it afterwards. The lineage of checkpoint datums is append-only, so a verifier of any later claim can walk digests backwards instead of trusting a prover-supplied tip.

Off-chain: the live fold proof, and the witness data. The latest recursive proof is kilobytes and self-verifying, so anyone can mirror it. The heavy item is not the proof but the witness-serving data: the actual Σ trees (UTxO SMT, epoch MMR, stake, reward, and governance trees, hundreds of gigabytes) that indexers keep so users can extract the Merkle paths their claim circuits open. Losing that data breaks nothing cryptographic (Ω stays valid, committed checkpoints stay verifiable), but new claim witnesses for the affected ranges cannot be built until someone re-derives the trees from chain data. That makes witness availability an operational concern (§11, data availability), not a trust concern.

9.4 Update cadence

(1) fold (S_block)	-- -- -- -- -- -- -- --	every block, ~20 s, trailing settlement
(1b) epoch fold (S_epoch)	-----o-----o--	every 5 days: rewards, snapshots, era changes
(2) settlement lag [tip immutable edge]		Praos k-depth: ~12 h Peras certs: minutes
(3) checkpoints (on-chain tx)	--o-----o-----o-----o----	policy: per epoch down to hourly; 1 tx each
(4) backfill	o	once, genesis to now

Four clocks run at different speeds:

1. **The fold advances every block.** One S_block step per ~20 s block is the operator's standing job; the full step (measured 3.5M-constraint header check plus body application, roughly 7M to 14M in total, §6) is why one GPU-class machine or a short CPU pipeline keeps up. S_epoch adds a burst every 5 days.
2. **The fold trails the settled prefix, not the tip.** IVC cannot be rolled back: if the fold consumed a block that a reorg later removed, the accumulator would fork and would have to be re-folded from the divergence point. So the fold only eats blocks that are already immutable. Under plain Praos that is the $k=2160$ depth (Cardano's rollback bound: blocks more than 2,160 below the tip cannot be reverted), roughly 12 hours behind the tip; with [Peras](#) certificates it drops to minutes. Settlement, not proving speed, bounds how fresh a provable fact can be, which is one more concrete reason Peras matters here.
3. **Checkpoint cadence is policy, not protocol.** Each on-chain checkpoint costs one transaction with one Groth16 verify (about 32% of a budget). Once per epoch is the natural floor, aligned with S_epoch; hourly or daily suits applications that consume recent facts on-chain. Laziness is cheap because historical claims never wait: any checkpoint covers all history before it, so a claim about epoch 300 verifies against last week's digest. Only claims about very recent events care about freshness.
4. **Nothing recurs for old history.** Committed checkpoints stay valid anchors forever, and blocks folded once are folded for good. The genesis backfill (§6) is a one-time fleet run; steady state is one machine, a 5-day burst, and a checkpoint transaction at whatever cadence consumers pay for.

9.5 Trusted-setup cadence

There is a fifth clock, the slowest of all: how often the Groth16 landing needs a trusted-setup ceremony. This is worth stating precisely because the naive answer, "Groth16 needs a per-circuit ceremony," makes the system sound far more fragile than it is. The right target, and an achievable one, is a ceremony that runs once and re-runs only on a deliberate proof-system change, on the same cadence and with the same coordination as a Cardano hard fork.

What actually triggers a ceremony. Groth16 setup has two phases with different triggers, and the split is verified at the code level in gnark's `mpcsetup`. Phase 1 (Powers of Tau, secrets tau/alpha/beta) is circuit-independent: you never run it, you inherit an existing public transcript such as the Perpetual Powers of Tau (good to 2^{28} constraints) or the Filecoin and Midnight lineages, and it is reusable for any circuit under its degree bound. Phase 2 (secrets delta and sigma) binds to the exact constraint system: gnark's `Phase2.Initialize` iterates every constraint, and snarkjs's `zkey verify` checks the key against the circuit. So

the only event that forces a fresh ceremony is a change to the landing circuit's own constraint system. Nothing else does: not a new Ω digest, not a new claim, not a new epoch, not more proofs.

Why you cannot remove this with a different Groth16. Groth16's reference string is irreducibly circuit-specific. The updatable-SRS impossibility result (Groth, Kohlweiss, Maller, Meiklejohn, Miers, 2018, eprint 2018/280) shows that a proof system with private secret-dependent polynomials in its reference string cannot be updatable, and updatability is the route to universality, so a string with those polynomials cannot be universal either. Groth16 has exactly those polynomials. Universality lives instead in the Plonk, Marlin, and Sonic family, whose reference strings use only secret-dependent monomials. So a rare ceremony cannot come from a smarter Groth16. It comes from freezing the one circuit that gets the ceremony.

The fix: a frozen wrap, changes absorbed by the recursion. The Omega landing already has the right shape. The Groth16 wrapper proves "a valid Halo2 chain-fold proof for Ω exists," with a single public input `pub = H(envelope ||  $\Omega$ )`. Everything that changes over Cardano's life must enter the wrap only through that value, never as new circuit wires: the Ω digest each checkpoint, which of the 42 claims, which epoch, and most of all which era's rules the fold applied. The era-dispatch logic lives inside the Halo2 fold, whose verifying key can change freely because Halo2 needs no ceremony at all. As long as the Halo2 proof's interface stays fixed (one public input, a fixed committed-instance layout, a fixed SRS degree bound), the wrap's constraint system is fixed and its verifying key is invariant.

This is not a hope; it is the production pattern behind RISC Zero, which verifies unbounded, ever-changing programs against one fixed Groth16 circuit and one verifying key. Its security model states the property directly: the identifier of the allowed inner logic (the "control root") is passed as a public input, "allowing for updates to our RISC-V Prover without requiring a new trusted setup ceremony." A router contract keeps even the caller-facing verifier address fixed while dispatching to versioned verifiers, so an eventual wrap change need not break consumers. RISC Zero ran its ceremony once (a public multi-contributor Phase 2 on top of a reused Phase 1), not once per program.

The resulting cadence. With the wrap frozen this way:

- new checkpoint, new claim, new epoch, more proofs: no ceremony;
- a Cardano hard fork that adds an era: no ceremony, because the new era's rules are new logic inside the fold, absorbed as witness and public-input data, not a change to the wrap;
- a new ceremony only when the wrap's own constraint system changes, which means a proof-system upgrade, a pairing change, or a landing redesign. That is a deliberate, coordinated event you would schedule exactly like a hard fork.

The zero-ceremony alternative remains on the table. Landing via the direct [Halo2-Plutus verifier](#) over the [CIP-0381](#) builtins removes the Groth16 wrap entirely. Halo2 rides a universal KZG setup, which for the Midnight side is already done once and for all (the Midnight ceremony to 2^{25} , finalized December 2025), so this route needs no Groth16 ceremony ever. The precedent is Zcash, which eliminated per-upgrade ceremonies outright when it moved to Halo2 at NU5. The trade is on-chain cost: MSM-dominated verification that the [CIP-0133](#) multi-scalar-multiplication builtin is designed to bring down. This is the same landing choice weighed in §11; the ceremony analysis does not decide it, because both endpoints reach hard-fork-or-

rarer cadence. The frozen Groth16 wrap gets there with the cheapest on-chain verification; the direct Halo2 landing gets there with no ceremony at all.

The practical instruction that falls out: treat the wrap circuit as a frozen artifact with the same "one relation, four descriptions" discipline the recovery circuit uses, because its immutability is now a security-relevant property rather than mere hygiene, and run its one ceremony the way RISC Zero did, inheriting Phase 1 and running a public Phase 2 once.

10. Roadmap

- **Phase 0, header-circuit spike: done** (§6). Real K for S_block: 22 (R1CS) / 24 (Plonkish, untuned).
- **Phase 1, consensus fold with an interim anchor:** implement the S_block IVC in the midnight-zk stack with Mithril-certified stake snapshots as witness anchors (skipping reward derivation). Concretely: the fold verifies each epoch's Mithril certificate in-circuit and takes from it the `stake_set` root that the VRF threshold check reads, standing in for the fold-derived value until Phase 2. Omega inherits Mithril's signer-set trust temporarily, but the pipeline, epoch MMR, and claim layer are real.
- **Phase 2, full state derivation:** add S_epoch (rewards, snapshots, era transitions) and the Byron prologue, then drop the Mithril anchor. Omega is now genesis-anchored. Run the backfill fleet.
- **Phase 3, claim layer:** the 42 claim circuits plus registry entries (`anchor_type = omega_accumulator`), per the existing [claim-interface-schema](#).
- **Phase 4, landing and self-checkpointing:** extend the Preview `groth16VerifyCommitted` validator with the digest-lineage datum; measure the Halo2-in-Groth16 wrap against the direct Halo2-Plutus verifier and pick one.

11. Open problems

1. **Reward-rule fidelity.** The Shelley reward calculation (spec figures 46 to 52), pool ranking, and its era-by-era patches must be mirrored in-circuit exactly. This is the largest spec-to-circuit surface. The "one relation, four descriptions" discipline from proof-zk-recovery (spec / circuit / on-chain / Lean) applies.
2. **Halo2-verifier-in-Groth16 cost.** Unmeasured (estimated K = 25 to 26); the fallback is the direct [Halo2-Plutus verifier](#).
3. **Chain selection in-circuit.** Model B needs a density/Genesis-style rule or an explicit statement that canonicity is pinned by Layer-4 self-checkpointing plus the honest-majority assumption. The exact formulation deserves its own page.
4. **Data availability for proving.** The backfill needs all ~220 GB with deterministic serialization (an archive node or Mithril DB snapshots as a data source only; no trust is inherited from them).
5. **Era-rule divergence.** Ten protocol versions of header, body, and ledger deltas (TPraos vs Praos VRF format, Alonzo `invalid_transactions`, Conway governance). Each is a circuit variant the IVC must dispatch on `era_id`.
6. **Poseidon parameterization** over BLS12-381 Fr for the Σ structures (choice, security margin, lookup-friendly alternatives such as Griffin or Anemoi). Pick once, early; it is baked into every root.

7. **Spike hardening.** The Phase 0 circuits are compile-measured but not test-vectored. Wire real header vectors (Koios) through witness solving before trusting the gadgets themselves.
8. **Operator incentives and checkpoint funding.** Who pays for continuous fold proving, Σ -tree witness hosting (§9.3), wrapping services, and the per-checkpoint transactions (§9.4), and through what mechanism (protocol or treasury funding, service fees, application sponsorship). The design is permissionless but not yet self-funding; products need a stated cost model before they can price their own UX.

12. The Midnight side: proving Midnight facts needs no fold

A bridge has two directions, and this report has so far described only one: proving Cardano facts on Midnight. The reverse direction, proving Midnight facts on Cardano, is the easier half, and the reason is a clean structural asymmetry.

Cardano forces the fold; Midnight hands you the roots. The whole Omega accumulator exists because a Cardano header commits no ledger-state root (§2), so the only way to prove a historical Cardano fact is to re-derive the state from genesis. Midnight is the structural opposite. Its ledger keeps every subsystem as an authenticated Merkle structure and commits the roots as first-class ledger state, so proving a Midnight fact is a membership proof against a committed root plus a finality certificate for the block that published it. No genesis replay, no accumulator. Confirmed committed roots, from the midnight-ledger specification:

- the **Zswap coin-commitment tree root**, with a rolling root-history window (`commitment_tree_history`), so membership validates against any recent root;
- **two DUST roots**, the UTxO commitment tree and the generation-info tree, each with a root history;
- **contract public-state roots**, since each contract's state is a Merklized trie with a native `root` operation, exposed to Compact as `MerkleTree` and `HistoricMerkleTree` with a `checkRoot` predicate;
- the **ledger and system parameters** (including DUST parameters) committed per block;
- the **replay-protection intent-history root**;
- a single **whole-ledger state_hash** over the content-addressed `LedgerState` DAG (parameters, Zswap, contract, UTxO, replay protection, DUST, pools), which every honest node computes identically.

One honest caveat: these roots are defined and committed as ledger state in the ledger specification, and the indexer surfaces them per block and per transaction, but the exact consensus block-header field that pins `state_hash` lives in the node and Substrate layer outside the ledger spec set. The roots exist and are committed; the header-level binding is strongly implied by the deterministic `state_hash` and the parent-hash threading, and confirming the precise field is a small follow-up.

Because Midnight is a Halo2/Plonkish chain over KZG on BLS12-381 with native in-circuit recursion and proof aggregation (the same [midnight-proofs recursion](#) the Cardano fold reuses), these membership statements are themselves provable and composable in-circuit with no external accumulator. A Midnight fact lands on Cardano through the bridge's Cardano-to-Midnight leg described in the [recursive bridge design](#); the same [claim envelope](#) carries it, with the anchor being a Midnight committed root and a Midnight finality certificate rather than the Ω digest.

12.1 What can be proven about Midnight

The Midnight predicate catalog counts 52 first-class claim predicates across eight groups. All 52 were validated against the midnight-ledger specification; each names a real Midnight mechanism. Two senses of "prove" coexist on Midnight and the catalog spans both: a **native ZK proof**, where a Compact/ZKIR, Zswap, DUST, or aggregation circuit proves a statement under a verifier key and public inputs, and a **historical or public-chain claim**, a fact about a block, transaction, contract state, or committed root that becomes succinct only when anchored to an authenticated root or finality certificate.

Group	Predicates	What it proves	Committed root or surface
Chain and indexer anchors	1 to 6	block existence and parent link, protocol version, ledger and system parameters, Zswap and DUST roots and end indexes, transaction inclusion and identity, transaction result and fee	block header, <code>ledgerParameters</code> , Zswap and DUST root histories
Transaction and intent validity	7 to 13	intent presence and hash, TTL liveness, transaction binding commitment and proof of exponent, unshielded-input signatures, canonical ordering and disjointness, segment sequencing, replay protection	intent history root, binding commitment
Contract state and calls	14 to 23	deployment address and zero initial balance, public state at an offset, registered verifier key per entry point, update authority, call proof and communication commitment, guaranteed versus fallible transcript, call context, and effect matching for Zswap, child calls, and mints	contract public-state root, <code>ContractState.operations</code>
Compact private predicates	24 to 30	private-state possession, secret-derived authorization, selective disclosure, range and threshold rules over hidden data, signature-backed off-chain attestation verified in-circuit, ledger ADT transitions, contract-local Merkle or historic-tree membership	native ZK, no on-chain anchor needed
Zswap shielded tokens	31 to 39	coin commitment from coin info and owner, inclusion in a historical root, nullifier derivation, nullifier nonmembership at apply time, spend authorization, output receive and ciphertext binding, Pedersen value-commitment correctness, per-segment balance conservation, transient create-and-spend	Zswap commitment tree root, nullifier set
Unshielded and NIGHT	40 to 43	transparent UTxO membership and unspent status, spend signature and intent binding, output creation and metadata time, unshielded token or contract balance and mint	UTxO state
DUST	44 to 50	registration signed, NIGHT-to-DUST delegation, generation-info membership and uniqueness, DUST commitment and non-double-spend, time-decayed available value from generation, cap, decay, and time, spend and same-owner rollover, fee payment within the grace period	DUST commitment and generation roots, DUST parameters
Aggregation and infrastructure	51 to 52	multiple inner statements verified under known verifier keys, and partner-chain committee, SPO, stake, or epoch-performance facts	midnight-zk aggregation crate, indexer partner-chain surface

The DUST group is the one with no Cardano analogue. DUST is Midnight's fee resource, and its value is not a fixed balance but a curve over time driven by the backing NIGHT, a generation rate, a cap, a decay rate, and a grace period, so a DUST claim binds to the DUST parameters and the declared action time, not just to a root. The Compact private-predicate group is the other one with no Cardano parallel, because it proves statements over private local state that never touches any chain, which is the whole point of Midnight's selective-disclosure model.

13. Provenance

- Measured circuit numbers: `spikes/omega-header-circuit/RESULTS.md` (2026-07-15, gnark v0.15.0, reusing proof-zk-recovery gadgets; private workspace, not in this repo).
- Deployed Groth16 verifier numbers: [Preview deployment](#) / [on-chain cost](#).
- Chain statistics (block heights, era boundaries, epoch/delegator/SPO counts, chain size): live Koios and AdaStat fetches, 2026-07-15, mainnet epoch 643.
- Header/CDDL structure (no state root; TPraos vs Praos VRF fields; Byron formats): IntersectMBO/cardano-ledger CDDL specs, fetched 2026-07-15.
- Claim catalog and envelope: [claim-interface-schema](#), [claim-envelope](#), [anchor-trust-models](#).
- Substrate and recursion context: [recursive bridge design](#), [midnight-proofs recursion](#), [halo2-plonkish](#), [CIP-0381](#), [CIP-0165](#).
- Trusted-setup cadence (§9.5), from a multi-agent literature and repo review, 2026-07-15: the two-phase MPC ceremony (Bowe, Gabizon, Miers, eprint 2017/1050) and its code realization in gnark `mpcsetup` and `iden3/snarkjs`; the amortization and 4x phase-2 cost figures and formal security (eprint 2025/064 SoK on powers-of-tau setups; eprint 2021/219 Snarky Ceremonies); the updatable-SRS impossibility for Groth16 (eprint 2018/280) and universality of Sonic/Marlin/Plonk (eprint 2019/099, 2019/1047, 2019/953); the fixed-wrap production pattern (RISC Zero security model and verifier docs, SP1); real-system ceremony cadence (Zcash NU5/Halo2, Filecoin, Ethereum KZG, Aztec Ignition); and the Midnight universal KZG setup to 2^{25} finalized December 2025.
- Midnight predicate catalog (§12): the 52-predicate catalog validated against the midnight-ledger specification (`spec/zswap.md`, `dust.md`, `intents-transactions.md`, `contracts.md`, `night.md`, `onchain-runtime.md`) and the indexer GraphQL schema; committed-root inventory from `LedgerState.state_hash` and the Zswap, DUST, and contract Merkle structures; Midnight proof-system context from the `midnight-zk proofs` and `aggregation crates`. All 52 mechanisms confirmed real; framing confirmed that Midnight commits state roots natively, unlike Cardano.
- Fact-check pass (2026-07-15): every measured circuit number in this report was reconciled against `spikes/omega-header-circuit/RESULTS.md` and the proof-zk-recovery README and matched exactly; the Cardano chain statistics were reconfirmed against live Koios and the ledger CDDL; the §9.5 ceremony claims were reconfirmed against their primary sources. Corrections applied: the §2 Shelley-family header label now distinguishes TPraos from Praos, and the §9.5 statement of the updatable-SRS impossibility result now matches the paper's exact wording (secret-dependent polynomials, impossibility of updatability).